Automated Processes Algorithmic Solution Report

Created by

MIDTOWN IT

by

Jaineel

PROJECT REFERENCE:

DATE:14/10/2025

Contents

PART 1: Specifying software application requirements and formulating automated solution 4

Automated process description 4

Client automation expectations 4

Expected functionality of automated processes 4

Security requirements 4

Required operating environment 4

Confirming application requirements 4

Document potential solutions 5

Potential solution discussion with the manager or team members 5

Confirm best solution with the manager 5

PART 2: Developing algorithmic solution. 6

Pseudocode algorithm 6

Explain the algorithm logic 6

Testing the algorithm – Desk Check Table 7

PART 3: Determining integrated development environment (IDE) and scripting language 8

Selecting Python framework and IDE 8

Translating pseudocode into scripts 8

Select the most efficient syntax elements and controls available in the selected scripting language. 8

Use the debugging features of the selected IDE to debug and test the scripts. 8

Create internal documentation (comments) according to organisational guidelines. 8

PART 4: Finalising scripts and obtaining sign-off 9

PART 5: Contingency tasks 10

Introduction 10

Contingency task 1 10

Contingency task 2 10

Appendix A: Communication log 11

PART 1: Specifying software application requirements and formulating automated solution.

This section details the initial phase of the project: specifying the software application requirements based on the Midtown IT asset audit scenario. A clear, definite, and documented set of requirements is necessary to guiding the developer, ensuring that the final Python script is relevant, secure, and meets all the required client expectations. The following subsections analyse the current operational scenario to define the process to be automated, the desired application outcomes, client expectations regarding functionality and security, and the necessary operating environment.

Automated process description

The core process to be automated is the collection of essential computer asset data, or "fingerprints," from all end-user or employee machines across the Midtown IT network, and the successive maintenance of a centralised asset inventory file. The current process is assumed to be manual, which is time consuming and prone to human error, particularly in a large scale audit required for the Incident Response Plan update.

The automated task involves:

1. **Execution:** The Python program must be executed successively on each computer via a USB drive or portable SSD

2. **Data Collection:** Gathering eight specific data points from each individual local machine: Computer Name, IP address, MAC address, Processor Model like i5, Ryzen 7, Operation System such as Windows or Linux, System time like 12:00 or 18:00, Internet connection speed like 500MB/s, and Active ports.

3. **Data Persistence:** Reading an existing centralized asset CSV file, comparing the currently collected data against the file, and performing an update such as modifying currently existing records or insert and/or add new record operation.

Client automation expectations

MidTown IT is expecting the automation solution to be provided in an efficient , secure and a repeatable way in order to audit the companies devices. Some key expectations is that the program will collect system data from the company devices and will automatically update the company's central data record, whilst eliminating the manual import that is currently being used which causes inconsistencies.

The client also expecting:

- To efficiently operate across multiple operating systems (OS) such as Windows and Linux without the need for additional setup

- Require little to no user interaction after the execution of the program, as it should function in an semi autonomous fashion.
- After completing present a clear output message to confirm a successful data collection and/or highlight any errors that may have occurred
- Maintain compliance with Midtown IT's internal Python Coding Style Guidelines and data handling policies

By implementing this automation, the organisation aims to improve their operational process, support Midtown ITs ongoing cyber security assessments, and simplify their current process of maintaining and accruing asset inventory.

Expected functionality of automated processes

The automated system should achieve:

1) Collect system information limited to computer name, IP address, MAC address, processor model, operating system(OS), system time, internet connection sleep and active network ports
2) Compare the collected results with the pre existing Csv file to decide if the device needs to record updated or added
3) Add new data even if it is a already registered device or an unlisted devices
4) Display any alerts or logs if any discrepancies are found between the recorded and current data
5) Maintain and save the current CSV file in a legible and standardised format compatible with Microsoft Excel, LibreOffice Calc or other spreadsheet like program
6) Should be able to operate both in online and offline system, which should allow data to be collect from stand alone devices whilst using removable media such as a usb or portable SSD

This functionality will ensure that Midtown Its computer assets remain up to date and represent accurate data, supporting and creating informed decisions in their system management and future cyber security planning.

Security requirements

Security is vital to the project. The program must be built with defensive programming to prevent the possible chances of data loss and/or corruption and to protect sensitive network information such as IP and MAC addresses. These should be applied to help:

- Unsure all inputs and system reposes are valid before writing to Csv files
- Implement a robust error handler to deal and to prevent crashes and or unintended overwrites
- To avoid exposing confidential system information, sanitise all data outputs
- Have permission to restrict the program to file write to the program directory only

- Ensure the process remains locally only not uploading it or transferring it externally

- Ensure a clear internal documentation to reduce the chance of introducing vulnerabilities can be introduced during code maintenance

By enforcing measure listed above, Midtown Its cybersecurity and privacy expectations can be met during the automated process.

Required operating environment

The automated solution will run on standard Midtown IT infrastructure as well as some legacy systems, that included both Windows 10, which has been discontinued and should be Windows 11 and modern Linux distributions such as Ubuntu 25.10 “Questing Quokka”, Debian 13, Fedora 43, Arch 6.16.8 and macOS 26 (Tahoe), legacy systems include Windows XP, Vista, 7 and 8.:

- Hardware: Standard office workstations/desktops and laptops equipped with a minimum 8GB of RAM, a modern multi core CPU such as Intel i5 of Ryzen 5 and above and USB gen 3.0 ports for the portable execution of the program

- Software: Python 3.11 or higher which should be installed locally, with built in access to libraries such as os, platform, csv, socket and subprocess

- IDE: Visual Studio Code (VSC) or PyCharm Community edition, configured for syntax checking, debugging and version control integration.

- Storage: Access to a shares or removable drive such as a USB stick and/or a SSD which should contain the CSV files.

Confirming application requirements

To: Ross [Project Manager]
From: Jaineel Khatri
Date: Tuesday 21 October 2025
Subject: Confirtmation of application Requirements- Computer Fingerprint Automation Project

Dear Ross,
Following up on our previous previous meeting regarding the Automated Computer fingerprint project, I am writing to confirm the finalised software application requirements before proceeding with possible algorithm design and implementation.

As discussed, the agreed requirements are as follows:

- The program will only collected the required system information, that includes and is limited to only; Computer Name, IP address, MAC address, Processor Model, Operating System, System time, Internet connection, Internet Speed, and Active ports.

- Data on the CSV file will be stored locally and will be maintained on the single portable storage

- When or if the program detects an existing entry for a device in the CSV file , it will compare it to existing entries and update where differences are found and add or update where releveant

- The program will run without requiring internet access, locally on Windows, Linux and macOS

- Midtown IT's Python Coding Style Guidelines will be followed for the code and internal documentation as well as defensive programming measure will be include

Please confirm the supplied requirements align with your expectations so I can begin with the possible algorithm designs and implementations to most effectively complete your required goals.

Kind Regards,
Jaineel Khatri

Potential solution discussion with the manager or team members

To: Ross [Project Manager]
From: Jaineel Khatri
Date: 21 October 2025
Subject: Proposed Algorithmic Solution – Computer Fingerprint Automation

Dear Ross,

Based on the previously confirmed requirements . I have outlined a possible algorithmic solution for the proposed project, Computer Fingerprint Automation.

Proposed Solution:

- On each individual device the python program will be executed locally on either a USB stick or a a portable SSD

- To collect system information, built in libraries such as os, socket, platform and subprocess will be used.

- All collected data will be temporarily formatted and written and stored locally to the CSV file on the portable SSD or USB stick

- The master CSV file will be read by the script , then it will compare the current device data against the master CSV file and will either update or append a new record if needed

- The user will be notified if there were to be discrepancies detected via a console message

- Any and all updates to the CSV will be validated and saved before securely terminating the program

All possible outcomes such as including duplicate records, missing fields and offline execution have been taken into account

Regards
Jaineel

PART 2: Developing algorithmic solution

This section will outline the design phase of the algorithmic design of the Computer Fingerprint Collector project. This stages purpose is to develop a clear, concise and logical pseudocode solution that will define how this program will collect, compare and update system data for all devices such as computers and laptops on the Midtown IT Network

A simplified version in the form of pseudocode can provide a representation of how the program's logic will work, which can allow the developer and project manager to confirm the solutions structure before converting it to Python code. The design should follow structured programming principles such as using control flow constructs such as sequences, selection and interaction. Accuracy, reliability and efficiency can be ensured by the algorithm whilst also maintaining compliance with Midtown IT’s security and code guidelines.

Structured Pseudocode:

```
BEGIN ComputerFingerprintCollector_GUI
   SET BASE_DIR
   IF program is running as EXE THEN
      SET BASE_DIR = folder of executable
   ELSE
      SET BASE_DIR = folder of script
   ENDIF
   SET DEFAULT_INVENTORY_PATH = BASE_DIR + "inventory.csv"
   SET FIELDS = {
      ComputerName,
      IPAddress,
      MACAddress,
      ProcessorModel,
```

```
    OperatingSystem,
    SystemTime,
    InternetSpeed,
    ActivePorts
}
INITIALISE GUI
CREATE main window with title
    "Midtown IT - Computer Fingerprint Collector"
CREATE inventory file path field
    default value = DEFAULT_INVENTORY_PATH
INITIALISE collectedData = empty dictionary
INITIALISE unsavedChanges = FALSE
INITIALISE isRunning = FALSE
CREATE checkbox variables for all fingerprint fields
SET ComputerName checkbox = TRUE
DISABLE ComputerName checkbox
DISPLAY buttons:
    - Run SELECTED
    - Run ALL
    - Save / Append Inventory
    - Clear
    - Exit
DISPLAY status text area
DISPLAY "Ready. Tick items, run collection, then Save/Append."
FUNCTION: LogStatus(message)
APPEND message to status text area
FUNCTION: SetRunningState(running)
SET isRunning = running
IF running == TRUE THEN
    DISABLE Run SELECTED button
    DISABLE Run ALL button
    DISABLE Save button
    DISABLE Clear button
ELSE
    ENABLE Run SELECTED button
    ENABLE Run ALL button
    ENABLE Save button
    ENABLE Clear button
ENDIF
FUNCTION: GetComputerName
READ system hostname
IF hostname is empty THEN
    RETURN "N/A"
ELSE
    RETURN hostname
ENDIF
FUNCTION: GetIPAddress
CREATE UDP socket
CONNECT socket to external address
READ local socket address as IP
CLOSE socket
IF IP exists AND is not loopback THEN
    RETURN IP
ELSE
    RETURN "N/A"
ENDIF
FUNCTION: GetMACAddress
READ MAC address as integer
```

```
CONVERT integer to 12-character hexadecimal string
FORMAT into MAC address with colons
RETURN formatted MAC address
FUNCTION: GetProcessorModel
READ processor model
IF value is empty THEN
   RETURN "N/A"
ELSE
   RETURN processor model
ENDIF
FUNCTION: GetOperatingSystem
READ system name
READ system release
READ system version
COMBINE values into one text string
IF result is empty THEN
   RETURN "N/A"
ELSE
   RETURN operating system string
ENDIF
FUNCTION: GetSystemTime
RETURN current date and time formatted as YYYY-MM-DD HH:MM:SS
FUNCTION: GetInternetSpeed
INITIALISE result = "N/A"
START background thread to run speed test
WAIT until timeout or thread completes
IF thread still running after timeout THEN
   RETURN "N/A"
ELSE
   RETURN measured download and upload speed
ENDIF
FUNCTION: ScanPortRange(startPort, endPort, timeout, openPorts, lock)
FOR port from startPort to endPort DO
   CREATE TCP socket
   SET timeout
   TEST connection to 127.0.0.1 on port
   CLOSE socket
   IF connection result indicates open THEN
      LOCK shared openPorts list
      APPEND port to openPorts
      UNLOCK shared openPorts list
   ENDIF
ENDFOR
FUNCTION: GetActivePortsFullScan
INITIALISE openPorts = empty list
INITIALISE lock
SET startPort = 1
SET endPort = 65535
SPLIT full port range into chunks
CREATE worker threads for each chunk
FOR each thread DO
   START thread
ENDFOR
FOR each thread DO
   WAIT for thread to finish
ENDFOR
SORT openPorts
IF openPorts is empty THEN
```

```
    RETURN "Open ports: none"
  ELSE
    RETURN formatted list of open ports
  ENDIF
  FUNCTION: CollectFields(selectedFields)
  INITIALISE data = empty dictionary
  SET data["ComputerName"] = GetComputerName()
  IF ComputerName is empty OR equals "N/A" THEN
    RETURN empty dictionary
  ENDIF
  FOR each field in selectedFields DO
    IF field is not ComputerName THEN
      CALL matching collector function
      STORE result in data[field]
    ENDIF
  ENDFOR
  RETURN data
  FUNCTION: ShowResults(data)
  DISPLAY "---- Collected Results ----"
  FOR each field in FIELDS DO
    IF field exists in data THEN
      DISPLAY field name and value
    ENDIF
  ENDFOR
  DISPLAY "----------------------------"
  BACKGROUND PROCEDURE: RunCollection(selectedFields, label)
  DISPLAY label + ": starting..."
  SET running state = TRUE
  CALL CollectFields(selectedFields) → data
  IF data is empty THEN
    DISPLAY "Error: ComputerName could not be collected. Nothing saved."
    SET running state = FALSE
    STOP
  ENDIF
  SET collectedData = data
  SET unsavedChanges = TRUE
  DISPLAY label + ": complete."
  DISPLAY collected results
  SET running state = FALSE
  EVENT: Run SELECTED Button Clicked
  READ checked boxes into selectedFields
  IF ComputerName not in selectedFields THEN
    ADD ComputerName to selectedFields
  ENDIF
  IF only ComputerName is selected THEN
    DISPLAY error message
    RETURN
  ENDIF
  START background thread to run RunCollection(selectedFields, "Run SELECTED")
  EVENT: Run ALL Button Clicked
  SET selectedFields = all FIELDS
  START background thread to run RunCollection(selectedFields, "Run ALL")
  FUNCTION: EnsureInventoryExists(path)
  IF file does not exist THEN
    CREATE CSV file
    WRITE header row using FIELDS
  ENDIF
  FUNCTION: LoadInventory(path)
```

```
    CALL EnsureInventoryExists(path)
    OPEN CSV file for reading
    LOAD all rows into records list
    RETURN records
    FUNCTION: SaveInventory(path, records)
    OPEN CSV file for writing
    WRITE header row
    WRITE all records
    CLOSE file
    FUNCTION: AppendRecord(records, collectedData)
    READ ComputerName from collectedData
    IF ComputerName is empty OR equals "N/A" THEN
        DISPLAY error
        STOP
    ENDIF
    CREATE newRecord with all fields set to "N/A"
    FOR each key and value in collectedData DO
        IF value is not empty THEN
            STORE value in newRecord[key]
        ENDIF
    ENDFOR
    SET newRecord["ComputerName"] = ComputerName
    APPEND newRecord to records
    EVENT: Save / Append Inventory Button Clicked
    IF collectedData is empty THEN
        DISPLAY error message
        RETURN
    ENDIF
    READ file path from inventory path field
    IF file path is not absolute THEN
        CONVERT to BASE_DIR relative path
    ENDIF
    CALL LoadInventory(path) → records
    CALL AppendRecord(records, collectedData)
    CALL SaveInventory(path, records)
    SET unsavedChanges = FALSE
    DISPLAY "New line added to inventory (no updates performed)."
    DISPLAY "Saved to: " + path
    EVENT: Clear Button Clicked
    SET collectedData = empty dictionary
    SET unsavedChanges = FALSE
    FOR each checkbox except ComputerName DO
        UNTICK checkbox
    ENDFOR
    DISPLAY "Cleared selections and data."
    EVENT: Exit Button Clicked
    IF unsavedChanges == TRUE THEN
        DISPLAY confirmation box
        IF user chooses No THEN
            RETURN to GUI
        ENDIF
    ENDIF
    CLOSE application window
END ComputerFingerprintCollector_GUI
```

Explain the algorithm logic

The Computer Fingerprint Collector is an event-driven application with a Graphical User Interface (GUI). Its design shifts from a linear, terminal-driven approach to one that responds dynamically to user input, such as button clicks and checkbox selections. This design enables users to initiate the data collection process for a single fingerprint item, multiple selected items, or the complete set of data.

The algorithm for Midtown IT's automated processes is designed for efficiency and accuracy. It uses selection control structures to process only the fingerprint items specifically chosen by the user via the Graphical User Interface (GUI), which minimizes unnecessary system calls. Following selection, a sequential flow gathers the selected system information and temporarily stores it.

To guarantee reliable asset tracking, the computer name is always collected as the crucial primary identifier for updating the asset inventory. Basic data validation is performed before any file operations begin to ensure all necessary information is present.

For saving results, the algorithm employs an iteration structure to scan the existing inventory file. If a device match is found, the corresponding record is updated; otherwise, a new record is added. This methodical approach ensures the asset register remains accurate and current.

Overall, the synergy of event-driven execution, selection-based processing, and controlled iteration results in an efficient, user-friendly, and highly suitable algorithm for the GUI-based automation solution.

Testing the algorithm – Desk Check Table

Input	Initial Assignment	Processing	Processing	Re-assignment	Output
User Action	**Selected Mode**	**Data Collection**	**Inventory Check**	**Record Action**	**Result**
Run ALL	selectedItems = ALL	Collect all fingerprint data	No matching Computer Name	New record created	New device added
PC-101 collectedData initialised	All 8 fields collected	inventory.csv exists	Append record	Success message	
	Validation passed	Match = False	Nothing to re-assign	Inventory updated	
Example with different input					

Run SELECTED	selectedItems = {IP, Ports}	Collect selected data only	Matching Computer Name found	Update selected fields	Record updated
PC-101 collectedData initialised	IP + Ports collected	inventory.csv exists	Update mode	Success message	
	Validation passed	Match = True	Fields updated	Inventory saved	
Example with different input					
Run ONE	selectedItems = {System Time}	Collect Computer Name + System Time	No matching Computer Name	New record created	New device added
PC-202 collectedData initialised	Missing fields allowed	inventory.csv exists	Insert mode	Success message	
	Validation passed	Match = False	Missing fields set to N/A	Inventory saved	
Example with error condition					
Save clicked	collectedData incomplete	Computer Name missing	Validation fails	Save blocked	Error message
	No data written	inventory.csv unchanged	Nothing to re-assign	Data protected	

PART 3: Determining integrated development environment (IDE) and scripting language

Selecting Python framework and IDE

Introduction

Python was chosen as the scripting language for the Midtown IT Computer Fingerprint Collector due to its suitability for automation tasks, cross-platform compatibility, and lightweight nature. Its standard library is capable of accessing both system and network information, which means data collection and CSV updates can be implemented with minimal reliance on external dependencies. This enhances portability, particularly when the program is run from removable media on various employee devices.

As a high-level, interpreted scripting language, Python is known for its strong emphasis on readability and rapid development. It supports multiple programming paradigms—including procedural, object-oriented, and functional—making it an excellent choice for both structured scripts and scalable applications. Python is frequently utilised in IT automation, cybersecurity scripting, data processing, and systems administration because of its capacity to interact effectively with operating system functions and networking utilities.

Python is a suitable programming language for this project due to its robust features, including a socket-based networking model that supports standard protocols like TCP/IP via its socket library. This is crucial for tasks such as obtaining IP addressing details and validating network connectivity and speed. Furthermore, Python allows for controlled, local execution of subprocesses to interact with system tools (e.g., checking open ports).

The language's fully object-oriented nature, where almost all elements (strings, lists, functions, etc.) are objects, simplifies data management. Specifically, the use of structured objects like dictionaries (key/value pairs) is beneficial for efficient storage, updating, and writing of system data to CSV files.

b) Communicate your IDE selection to the manager or relevant personnel and seek confirmation. Include a detailed justification for your selection. The communication can take the form of an email.

To**:** Ross [Project Manager]
From: Jaineel Khatri
Date: 23 October 2025
Subject: IDE Confirmation – Python Development for Computer Fingerprint Collector

Dear Ross,

The proposed IDE for the Midtown IT Computer Fingerprint Collector project is VS Code.

This selection is based on several key benefits:

- **Strong Python Support:** VS Code offers robust Python support via extensions, including syntax validation, linting, integrated terminal capabilities, and advanced debugging features like breakpoints.
- **Alignment with Standards:** Its support for version control integration and clear project structure directly adheres to Midtown IT's coding style expectations and documentation requirements.
- **Cross-Platform Suitability:** Being lightweight and consistently usable across various operating systems, VS Code meets the project's requirement for cross-platform development and testing.
- **Efficient Debugging:** The IDE's debugging tools will be crucial for the efficient identification of logic errors, particularly during CSV updating, selection logic (Run One/Selected/All), and validation steps.

I request confirmation of this IDE selection to allow implementation and testing phases to commence.

Kind regards,
Jaineel Khatri

Translating pseudocode into scripts

Select the most efficient syntax elements and controls available in the selected scripting language.

Python was chosen as the scripting language due to its efficient syntax elements and controls, which are highly readable, structured, and closely resemble pseudocode. The script utilises efficient syntax elements and controls, including variables, lists, dictionaries, functions, selection statements (if), iteration statements (for), and classes. The efficiency of these elements lies in their ability to minimise code repetition, simplify program maintenance, and facilitate the organised collection, validation, and storage of data.

A part of the script that includes basic language elements of data is:

```
FIELDS = [
    "ComputerName",
    "IPAddress",
    "MACAddress",
    "ProcessorModel",
    "OperatingSystem",
    "SystemTime",
    "InternetSpeed",
    "ActivePorts",
]
```

This section illustrates fundamental data language elements, specifically:

- **Strings:** Examples include "ComputerName" and "IPAddress," which are used to label fingerprint data fields.
- **List and Variable:** A list named FIELDS is employed to store these multiple values in a structured format, with the list itself being assigned to a variable of the same name.

This approach is highly efficient as centralizing all fingerprint field names simplifies program logic when referencing them, whether for collecting system data or writing records to the CSV inventory file.

The append_reccord(records, collected) function contains a script section that incorporates both selection and iteration.

```
def append_record(records, collected):
    name = (collected.get("ComputerName") or "").strip()
    if not name or name == "N/A":
        raise ValueError("Computer Name is required.")

    new_record = {k: "N/A" for k in FIELDS}
    for k, v in collected.items():
        if v not in ("", None):
            new_record[k] = v
    new_record["ComputerName"] = name

    records.append(new_record)
```

Iteration is implemented using a for loop, which processes each key-value pair in the collected dictionary. This dictionary contains the gathered system fingerprint data.

Selection is handled by an if statement within the loop. This statement checks if the current value is neither empty nor None. Only if this condition is true is the value assigned to the new_record dictionary.

This design was chosen for its efficiency, as it avoids repetitive code for each fingerprint field. Instead of checking every field individually, the loop automatically processes all collected data. The selection mechanism is crucial for ensuring that only valid values are stored in the record before it is added to the inventory list. This approach enhances code readability, minimizes duplication, and simplifies program maintenance.

Use the debugging features of the selected IDE to debug and test the scripts.

The Python script was developed and thoroughly debugged using Visual Studio Code (VS Code), leveraging its Python extension and integrated debugging tools. These tools were essential for identifying issues, monitoring variable values, and confirming proper program execution.

Initial development involved testing the script directly within the IDE. Breakpoints were strategically placed in key functions, such as collect_fields() and append_reccord(). The Step Over debugging feature was used to execute the code line by line, allowing close inspection of variable values via the Variables panel and the Debug Console.

A critical issue was identified during testing of the compiled executable version: popup windows would momentarily appear and disappear upon launching the application. Investigation revealed that this unexpected behavior was caused by the incorrect packaging of external third-party libraries originally used for collecting the IP and MAC addresses during the conversion of the Python script into an executable file.

To resolve this, the problematic external libraries were replaced with built-in Python modules: the socket module for retrieving the IP address and the uuid module for the MAC address. Following this modification, the program executed correctly in both the IDE environment and as a stand-alone compiled executable.

Final comprehensive testing was then conducted to verify all functions operated as intended and that the GUI application successfully collected fingerprint data and saved it to the CSV inventory file.

Testing Report

The following steps detail the testing process used for the script:

Debugging and Variable Confirmation:

1. The script was opened in Visual Studio Code.
2. A breakpoint was set within the append_reccord() function.
3. The debugger was initiated using Run and Debug.
4. Execution paused at the breakpoint.
5. Program behavior was confirmed by checking variable values in the Variables panel.

GUI and Functionality Testing:

1. The GUI application was tested by selecting fingerprint items and clicking either Run ALL or Run SELECTED.
2. The collected results were reviewed in the GUI's status panel.
3. The Save / Append Inventory button was used to write the collected data to inventory.csv.

Verification and Final Confirmation:

1. The generated CSV file was opened and checked to verify the record was saved correctly.
2. After resolving a library issue, the program was retested to confirm correct execution as both a Python script and an executable file.

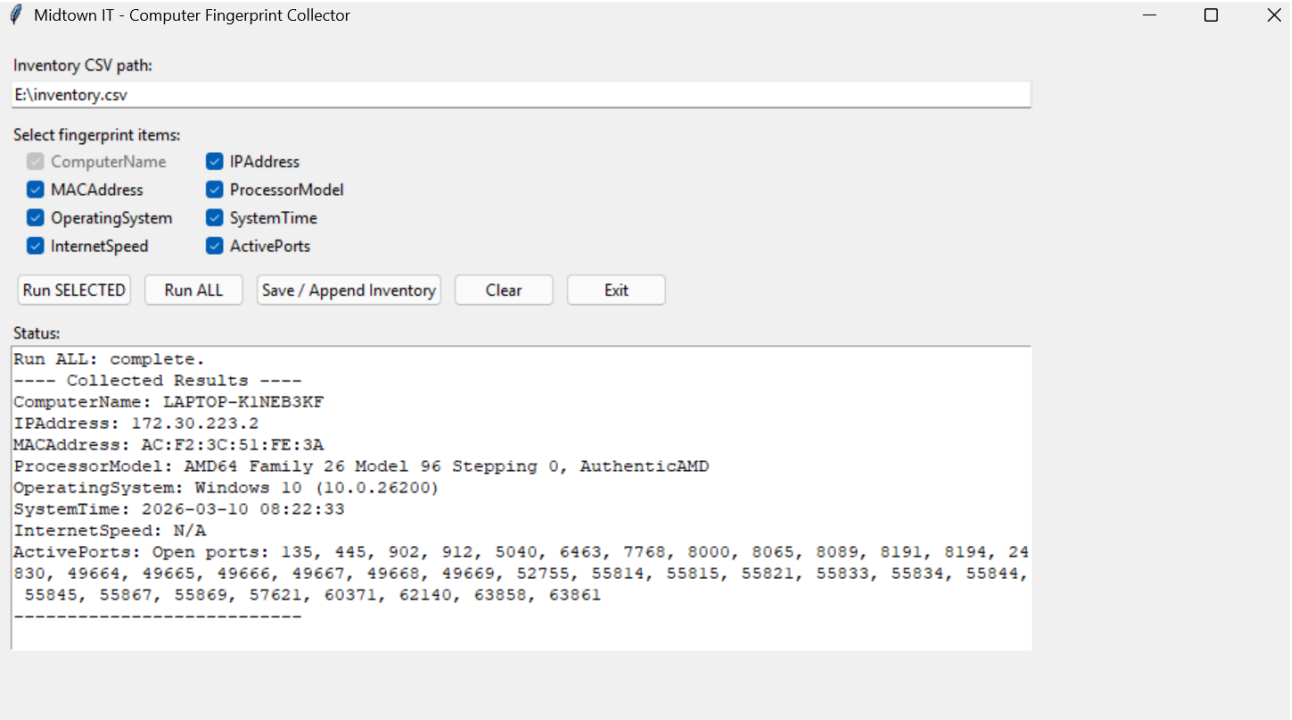
The following table summarises the results of the application's testing:

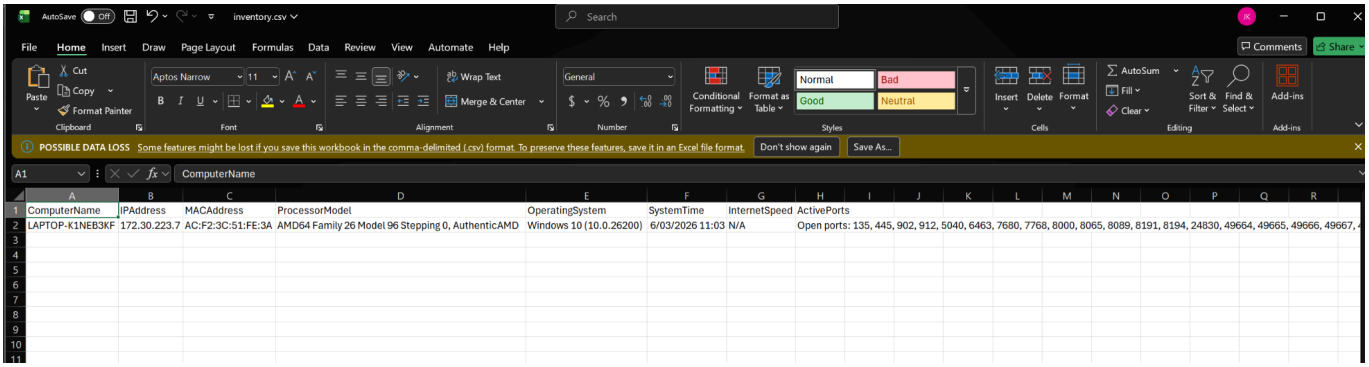
Test No.	Test Description	Expected Result	Actual Result	Status
1	Launch the program GUI in VS Code.	GUI opens successfully.	GUI opened correctly.	Pess
2	Set a breakpoint in the append_reccord() function and run the debugger.	Program pauses at the breakpoint.	Debugger paused correctly.	Pass
3	Inspect variable values during the debugging process.	Variables display correct values.	Values displayed correctly in Variables panel.	Pass
4	Run the SELECTED process with chosen fingerprint items.	Selected items are collected and displayed.	Data displayed in status panel.	Pass
5	Run the ALL process.	All fingerprint fields are collected.	All data displayed correctly.	Pass
6	Execute the Save / Append Inventory function.	Collected data is saved into inventory.csv.	Record saved correctly.	Pass
7	Verify the saved CSV file content.	Header row and the newly saved record are visible.	CSV file contained correct data.	Pass
8	Test the executable version following code corrections.	Program runs correctly as an .exe file.	Program worked correctly after replacing external library usage.	Pass

Screenshots Provided as Evidence:

```python
211 def append_record(records, collected):
212     name = (collected.get("ComputerName") or "").strip()
213     if not name or name == "N/A":
214         raise ValueError("Computer Name is required.")
215
216     new_record = {k: "N/A" for k in FIELDS}
217     for k, v in collected.items():
218         if v not in ("", None):
219             new_record[k] = v
220     new_record["ComputerName"] = name
221
222     records.append(new_record)
223
```

```python
210
211 def append_record(records, collected):
212     name = (collected.get("ComputerName") or "").strip()
213     if not name or name == "N/A":
214         raise ValueError("Computer Name is required.")
215
216     new_record = {k: "N/A" for k in FIELDS}
217     for k, v in collected.items():
218         if v not in ("", None):
219             new_record[k] = v
220     new_record["ComputerName"] = name
221
222     records.append(new_record)
223
```





ComputerName	IPAddress	MACAddress	ProcessorModel	OperatingSystem	SystemTime	InternetSpeed	ActivePorts
LAPTOP-K1NEB3KF	172.30.223.7	AC:F2:3C:51:FE:3A	AMD64 Family 26 Model 96 Stepping 0, AuthenticAMD	Windows 10 (10.0.26200)	6/03/2026 11:03	N/A	Open ports: 135, 445, 902, 912, 5040, 6463, 7680, 7768, 8000, 8065, 8089, 8191, 8194, 24830, 49664, 49665, 49666, 49667,

During the testing and debugging phase, several limitations and potential issues were noted in the script.

Identified Limitations (Omissions)

The current program version lacks full implementation in the following areas:

- **Network Scanning:** The port scanning function is limited to the local machine and does not extend to other devices on the network.
- **File Path Validation:** While the user can manually input the path for the CSV inventory file, the program does not adequately validate this path before attempting file read/write operations.
- **User Feedback:** There is no progress indicator displayed during long-running operations, such as internet speed testing or port scanning.
- **Error Management:** The script lacks a dedicated error logging system to record errors to a log file for subsequent troubleshooting.

Identified Issues (Errors)

The following issues were discovered during development and testing:

- **Library Dependency Conflict:** An earlier script version utilized external libraries for retrieving IP and MAC addresses. These libraries caused brief, disruptive popup windows when the Python script was compiled into an executable. This was resolved by switching to built-in Python modules(socket and uuid).
- **Unhandled Error on Missing Data:** The script raises a ValueError if the Computer Name cannot be retrieved, preventing the record from being saved.
- **Ambiguous Internet Speed Result:** If the internet connection is unavailable or the speed test times out, the result defaults to "N/A," which does not clearly explain the reason for the failure.

These issues and limitations were documented during the debugging and testing process using Visual Studio Code.

Create internal documentation (comments) according to organisational guidelines.

Internal documentation, in the form of comments, was integrated into the Python script to clarify the function of various program sections. These comments are crucial for script maintainability and future modifications, as they aid other developers in understanding the code's operation.

The comments adhere to established organisational coding guidelines, specifically:

- Clearly describing the purpose of a section of code.
- Being positioned above functions or code blocks to explain their behaviour.
- Utilising simple, clear language to ensure easy comprehension by other developers.

Examples of the comments used in the script are provided below:

```
# Set the base directory for the program.
# If running as a compiled EXE, use the executable folder.
# Otherwise use the folder where the Python file is stored.
if getattr(sys, "frozen", False):
    BASE_DIR = os.path.dirname(sys.executable)
else:
    BASE_DIR = os.path.dirname(os.path.abspath(__file__))

# Set the default path for the CSV inventory file
DEFAULT_INVENTORY_PATH = os.path.join(BASE_DIR, "inventory.csv")

# Define the list of fingerprint fields used throughout the program
FIELDS = [
    "ComputerName",
    "IPAddress",
    "MACAddress",
    "ProcessorModel",
    "OperatingSystem",
    "SystemTime",
    "InternetSpeed",
    "ActivePorts",
]
```

```python
# Worker function for speed testing.
# Runs in a separate thread so the GUI does not freeze.
def _speedtest_worker(result_holder):
    st = speedtest.Speedtest()
    st.get_best_server()
    download = st.download() / 1_000_000
    upload = st.upload() / 1_000_000
    result_holder["value"] = f"Down {download:.1f} Mbps, Up {upload:.1f} Mbps"


# Run the internet speed test with a timeout.
# If the test takes too long, return N/A.
def get_internet_speed(timeout_sec=20):
    result_holder = {"value": "N/A"}
    t = threading.Thread(target=_speedtest_worker, args=(result_holder,), daemon=True)
    t.start()
    t.join(timeout=timeout_sec)

    if t.is_alive():
        return "N/A"
    return result_holder.get("value") or "N/A"
```

```python
# Add a new fingerprint record to the list of inventory records
def append_record(records, collected):
    name = (collected.get("ComputerName") or "").strip()

    # Computer name is required to identify the record
    if not name or name == "N/A":
        raise ValueError("Computer Name is required.")

    # Start with all fields set to N/A
    new_record = {k: "N/A" for k in FIELDS}

    # Replace default values with collected values where available
    for k, v in collected.items():
        if v not in ("", None):
            new_record[k] = v

    new_record["ComputerName"] = name

    # Append the completed record to the records list
    records.append(new_record)
```

Identify Item Manipulation

```python
# Add a new fingerprint record to the list of inventory records
def append_record(records, collected):
    name = (collected.get("ComputerName") or "").strip()

    # Computer name is required to identify the record
    if not name or name == "N/A":
        raise ValueError("Computer Name is required.")

    # Start with all fields set to N/A
    new_record = {k: "N/A" for k in FIELDS}

    # Replace default values with collected values where available
    for k, v in collected.items():
        if v not in ("", None):
            new_record[k] = v

    new_record["ComputerName"] = name

    # Append the completed record to the records list
    records.append(new_record)
```

The script performs item manipulation by altering and adding items within data structures. Specifically, new_record[k] = v updates items in the new_record dictionary, assigning collected values to the corresponding field names. The line records.append (new_record) then incorporates the completed dictionary as a new item into the records list.

This Python-specific syntax was chosen for its efficiency. A dictionary is an ideal structure for storing fingerprint fields using a key-value format, which significantly enhances data readability and simplifies updates. Moreover, using .append() to add a new record to the list is efficient, avoiding the need to manually rewrite the entire list. This methodology reduces code repetition, improves readability, and contributes to easier script maintenance.

The final Python script MUST be submitted with the report

PART 4: Finalising scripts and obtaining sign-off

User-level documentation (how to use the application)

The Midtown IT Computer Fingerprint Collector, a portable, GUI-based Python application, is complete. It is designed to be executed locally on each workstation or laptop during the organisation's asset audit. For a self-contained and repeatable process, the application should be run from approved removable media (USB/portable SSD) alongside the central **inventory.csv** file. This method eliminates the dependency on external infrastructure for storing the results.

Purpose (what the tool does)

The application collects a standardized "fingerprint" from each local machine and centrally stores this information in a CSV inventory.

The collected fingerprint data consists of the following fields:

- **ComputerName** (Mandatory, functions as the primary unique identifier)
- IPAddress
- MACAddress
- ProcessorModel
- OperatingSystem
- SystemTime
- InternetSpeed
- ActivePorts

Pre-Requisites

The following conditions must be met before running the executable:

1. The endpoint device must be running Windows.
2. The file fingerprint_collector.exe must be located on approved removable media (USB or portable SSD).
3. The removable media must allow write access so the inventory.csv file can be created or updated.
4. The user must have standard permission to execute applications.

Python installation is NOT required. No software installation is required on the endpoint device.

How to Run the Application

1. Insert the approved USB or portable SSD into the workstation.
2. Open the folder containing fingerprint_collector.exe.
3. Double-click fingerprint_collector.exe.
4. The GUI titled “Midtown IT – Computer Fingerprint Collector” will open.
5. Confirm the Inventory CSV path is correct.
6. Select required fingerprint items using the checkboxes.
7. Click:
    - “Run SELECTED” to collect selected items, or

    - “Run ALL” to collect all fingerprint data.
8. Review the collected results displayed in the Status window.
9. Click “Save / Append Inventory” to add the record to inventory.csv.
10. Move to the next workstation and repeat the process

If inventory.csv does not already exist, it will be created automatically.

Expected Behaviour

When operating correctly:

- A “starting…” message appears.
- Data collection completes successfully.
- Results are displayed in the Status window.
- Clicking Save adds a new row to inventory.csv.
- A confirmation message displays the save location.

Error Handling

The application includes defensive error handling:

- If Internet Speed cannot be measured, the value will display as N/A.
- If Active Ports cannot be detected, the output will display “Open ports: none”.
- If any fingerprint item cannot be collected, it will return N/A.
- If Computer Name cannot be collected, saving is blocked to prevent unidentified records.

The application does not crash if the device has no internet connection.

Inventory Design Decision

The inventory system operates using an append-only model:

- Each Save action adds a new row.
- Existing records are not overwritten.
- Historical records are preserved.
- This supports audit traceability and prevents accidental data loss.

Testing Summary

The executable was tested under the following conditions:

- With internet connection
- Without internet connection
- Using Run SELECTED
- Using Run ALL
- Saving multiple records
- Exiting with unsaved changes

Testing confirmed:

- The GUI remains responsive during data collection.
- Records are appended correctly to the CSV file.
- The executable runs without requiring Python.
- Error conditions return N/A rather than causing application failure.

Python Code SIGN-OFF Signing off on this document signifies that the Python code and documentation presented complies with the client's business requirements and expectations.	
Project Manager - Ross Wang Signature: RossW Date: 10/03/2026	**Software Developer - Jaineel Khatri** Signature: JaineelK Date: 24/02/2026
Python code APPROVED Provide feedback regarding changes required.	
APPROVAL ☑ ~~Granted~~ ☐ Not Granted	

If necessary, update code as requested and re-submit for approval.

PART 5: Contingency tasks

Contingency task 1

Assume that you have completed the algorithm and translated it into the corresponding Python code. At this point, the client requests the addition of a couple of minor changes to the program. Outline the procedure you would follow to control and incorporate the changes.

When minor changes are requested after program completion, a controlled change management procedure must be followed to maintain compliance and ensure correct implementation.

This process involves the following steps:

1. **Document and Clarify Requirements:** The requested changes must first be documented and fully clarified with the client. This ensures an exact understanding of the requirements, preventing misunderstandings, and confirming that the changes are achievable without impacting existing program functionality.
2. **Review and Assess Impact:** The proposed changes are then reviewed and assessed to determine their impact on the current algorithm, program structure, and system performance. If necessary, the algorithm and pseudocode should be updated *before* modifying the Python script.
3. **Implement Changes:** The approved changes are then **i**mplemented in the source code within the development environment. Crucially, the modified sections of the script must be clearly commented to enable other developers to easily identify where changes were made.
4. **Test and Debug:** After implementation, the program must be tested again using debugging tools. This confirms that the new features function correctly and that no new errors or regressions were introduced by the changes.
5. **Finalise and Deploy:** The updated version is then documented and saved using version control or file versioning procedures. The client is informed of the completion, and the updated application can then be deployed.

This structured process is critical for managing program changes in a controlled manner, thus upholding software quality and adherence to organisational policies.

Contingency task 2

Assume that you have completed the Python script, and the client informs you that although the development was completed for a specific IDE version, a newer version including more features has recently become available. The company wants to deploy the application using the latest version. Explain how you would proceed considering the new client requirement.

If a client requests deployment using a newer version of the Integrated Development Environment (IDE), the initial action involves reviewing the compatibility of the existing Python script with the updated environment. The project was developed using Visual Studio Code, which is already a robust and widely-used IDE for Python.

While Visual Studio Code is a standard development environment, a newer version may offer enhancements, security updates, or additional features. Consequently, the script must be opened and tested in the updated IDE to confirm that all functions, libraries, and extensions remain fully operational.

Following this, the program should be executed and debugged using the new IDE version to verify that all functionality, including the graphical user interface, system information collection, and CSV file generation, performs as expected.

Should the newer IDE version introduce compatibility issues or errors, these problems must be documented and discussed with the relevant personnel. If necessary, the organization might temporarily continue to use the current stable version of Visual Studio Code until a resolution is found.

This methodology ensures the program's stability is maintained while still allowing the organization to benefit from the improvements offered by the newer IDE version.

Appendix A: Communication log

Communication Log				
Date	**Topic**	**Discussion**	**Stakeholder**	**Communication Type**
21/10/25	Confirming application requirements	Discussing if all requirements I have said meet expectations	Ross & Jaineel	Email
21/10/25	Potential solution discussion with the manager or team members	Discuss the solution to the problem and discussing a solution	Ross & Jaineel	Email
03/02/26	Code review with project	Discuss the code I have present with the Project Manager and review code, and get feedback to remove the use of try and remove some GUI elements and not to use some shorthand methods	Ross and Jaineel	Face to Face